

Cyber Threats and Vulnerabilities 1

Identify and Analyze Cyber Threats:

A. Malware Sample Analysis Using VirusTotal

3 / 72
Community Score -58

3/72 security vendors flagged this file as malicious

6740023be829f84fa543ebfb2f745e33cba576ed8f51d4fdb4331dc36c16a3b7
OLEPRX.dll
Size: 56.50 KB
Last Analysis Date: 56 minutes ago
DLL

DETECTION DETAILS RELATIONS BEHAVIOR COMMUNITY 3

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Basic properties

MD5: 2dcebe31f60be3a1edcdc37558e75f2
SHA-1: 94c080655148d278c7998a7e43f0da08753166bb
SHA-256: 6740023be829f84fa543ebfb2f745e33cba576ed8f51d4fdb4331dc36c16a3b7
Vhash: 1540666d5d1515151b24=21e
Authentihash: c06b9ace612ecf625d24cc085573008483a139c1366c550d9811b3a05fe27d4
Imphash: 20950c93f21e57e75716f301cc245775
Rich PE header hash: b18558b9c60bf2500425e2a1bd4fc0c
SSDEEP: 1536:8gs5VW8P5SzyRUJD61egrg1QGUvvwCCw+0XPH6UISDQJ:8gy7P5S2PXP6U0J
TLSH: T187432906A1A531BCC05E803046574F23A5B7C0855736CF70BA1FE793FF5528A27A89E
File type: Win32 DLL
Magic: PE32+ executable (DLL) (GUI) x86-64, for MS Windows
TrID: Win64 Executable (generic) (44.6%) Windows Icons Library (generic) (14%) OS/2 Executable (generic) (13.8%) Generic Win/DOS Executable (13.7%) DOS Executable (generic) (13.7%)
DetectItEasy: PE64 Compiler: Microsoft Visual C/C++ (19.36.37032) [C] Linker: Microsoft Linker (14.36.37032) Tool: Visual Studio (2022 version 17.6)
Magika: PEBIN
File size: 56.50 KB (57856 bytes)

B. Creation of 1 Phishing Template Using Social Engineering Toolkit (SET)

Sign in to GitHub · GitHub

Sign in to GitHub

Username or email address
victim

Password

Forgot password?

Signing in...

New to GitHub? Create an account

Terms
Privacy
Docs

Contact GitHub Support
Manage cookies

Do not share my personal information

set:webhacker IP address for the POST back in Harvester/Tabnabbing [10.0.2.15]: 10.0.2.15
[-] SET supports both HTTP and HTTPS
[-] Example: http://www.thisisafakesite.com
set:webhacker Enter the url to clone: https://github.com/login
[*] Cloning the website: https://github.com/login
[*] This could take a little bit...
The best way to use this attack is if username and password form fields are available. Regardless, this captures all POSTs on a website.
[*] The Social-Engineer Toolkit Credential Harvester Attack
[*] Credential Harvester is running on port 80
[*] Information will be displayed to you as it arrives below:
10.0.2.15 - - [18/Mar/2026 16:40:41] "GET / HTTP/1.1" 200 -
[*] GET / HTTP/1.1: Printing the output:
PARAM: commit=Sign-in
PARAM: authenticity_token=tCf0Z0jYraqcEh+LCRNa+HYLxyURFmTr0Xg+LR25RUQv4FygZy9HKDK5o40z5AYVRN1gCTK/048EzyFAzNx28w==
PARAM: add_account=
POSSIBLE USERNAME FIELD FOUND: login=victim
POSSIBLE PASSWORD FIELD FOUND: password=victimpassword1234
PARAM: webauthn-conditional=undefined
PARAM: javascript-support=true
PARAM: webauthn-support=unsupported
PARAM: webauthn-luvpaa-support=unsupported
POSSIBLE USERNAME FIELD FOUND: return_to=https://github.com/login
PARAM: allow_signup=
PARAM: client_id=
PARAM: integration=
PARAM: required_field_ad02=
PARAM: timestamp=1773866218503
POSSIBLE PASSWORD FIELD FOUND: timestamp_secret=8207e0369a17a1978d0b44dc5290450f62a3f0e430d5d150a25d02861c45f61301
[*] WHEN YOU'RE FINISHED, HIT CONTROL-C TO GENERATE A REPORT.
10.0.2.15 - - [18/Mar/2026 16:47:33] "GET /favicon.ico HTTP/1.1" 404 -

C. Mapping of 1 Real APT Campaign to MITRE ATT&CK Framework

Campaign: SolarWinds Compromise

Threat Actor: APT29

Source: <https://attack.mitre.org/campaigns/C0024/>

 = My Writing

 = Directly Quoted Text

ATT&CK Mapping:

1. T1059.001 - Command and Scripting Interpreter: PowerShell

“During the SolarWinds Compromise, APT29 used PowerShell to create new tasks on remote machines, identify configuration settings, exfiltrate data, and execute other commands.”

PowerShell fits under command and scripting interpreters because APT29 uses it to run arbitrary commands on a system.

2. T1560.001 - Archive Collected Data: Archive via Utility

“During the SolarWinds Compromise, APT29 used 7-Zip to compress stolen emails into password-protected archives prior to exfiltration; APT29 also compressed text files into zipped archives.” 7-Zip fits because it’s a third-party utility that APT29 uses to package collected data in order to make it smaller, easier to move, and less obvious.

3. T1568 - Dynamic Resolution

“During the SolarWinds Compromise, APT29 used dynamic DNS resolution to construct and resolve to randomly-generated subdomains for C2.” This fits because APT29 dynamically establishes connections to command and control infrastructure instead of relying on fixed and easy to block addresses.

4. T1018 - Remote System Discovery

“During the SolarWinds Compromise, APT29 used AdFind to enumerate remote systems.” This fits because APT29 was looking for other systems by hostname, IP address, or similar identifiers using AdFind to learn what was present in the environment.

5. T1053.005 - Scheduled Task/Job: Scheduled Task

“During the SolarWinds Compromise, APT29 used scheduler and schtasks to create new tasks on remote hosts as part of their lateral movement. They manipulated scheduled tasks by updating an existing legitimate task to execute their tools and then returned the scheduled task to its original configuration. APT29 also created a scheduled task to maintain SUNSPOT persistence when the host booted.” This fits because APT29 used scheduled tasks to run payloads automatically and maintain persistence. It fits the definition of “a way to run malicious code at a specific time or repeatedly.”

6. T1553.002 - Subvert Trust Controls: Code Signing

“During the SolarWinds Compromise, APT29 was able to get SUNBURST signed by SolarWinds code signing certificates by injecting the malware into the SolarWinds Orion software lifecycle.” This fits under the definition of defense evasion because APT29 is abusing the system’s trust model to make malicious software look legitimate. This was done through code signing abuse which helped it blend in and bypass trust-based controls.

7. T1546.003 - Event Triggered Execution: Windows Management Instrumentation

Event Subscription

“During the SolarWinds Compromise, APT29 used a WMI event filter to invoke a command-line event consumer at system boot time to launch a backdoor with rundll32.exe.” This fits the definition of persistence because APT29 used WMI event filters to launch the backdoor automatically when a trigger occurred. In this case it was when the system booted so the APT would survive a reboot.

8. T1016.001 - System Network Configuration Discovery: Internet Connection

Discovery

“During the SolarWinds Compromise, APT29 used GoldFinder to perform HTTP GET requests to check internet connectivity and identify HTTP proxy servers and other redirects that an HTTP request travels through.” This fits the definition of system network configuration discovery because APT29 checked whether the victim environment had internet access before moving tools or communicating externally.

Apply Vulnerability Assessment Techniques:

Asset Discovery Scan:

Step 1: Put Kali Linux and a target VM on a host-only network and write down the subnet.

Step 2: Find any live hosts by running:

```
nmap -sn 192.168.56.0/24 -oA asset_discovery
```

Step 3: Identify services on the discovered hosts by running:

```
nmap -sV -O <target-ip> -oA services_<target-ip>
```

Step 4: Perform basic network mapping by building a table with:

- IP addresses
- Hostnames
- Open ports
- Services
- Notes for devices (ex. “web server”)

Step 5: Explain the methodology used (written above) and the potential security implications.

Vulnerability Scan:

Step 1: In Nmap, run:

```
nmap -sV --script vuln <target-ip> -oA vuln_scan_<target-ip>
```

Step 2: Classify each vulnerability as Critical, High, Medium, or Low based on:

- How exposed the service is
- How likely to be attacked
- What the impact would be
- Whether or not the vulnerability affects confidentiality, integrity, or availability

Step 3: Write a summary of findings that shows:

- The methods used
- What systems were discovered
- What services/ports were exposed
- Any potential vulnerabilities
- Which vulnerabilities are the most critical
- What the security implications are

Implement Threat Intelligence Principles:

Analyze 2 IoCs:

Step 1: For IoC #1, put a SHA256 file hash into VirusTotal and record:

- The hash value
- Where it came from
- What VirusTotal showed
- Write reasons for why that suggests threat activity

Step 2: For IoC #2, put the domain or IP into VirusTotal and record:

- The domain or IP
- What the report from VirusTotal showed
- Why it's suspicious
- A short writeup that explains why the findings indicate threat activity and the security implications

OpenCTI Implementation:

Step 1: 1. Install OpenCTI using Docker.

Step 2: Start OpenCTI and confirm that the web UI is reachable.

Step 3: Create a dedicated user/token for each connector. Each connector needs OPENCTI_URL and OPENCTI_TOKEN.

Step 4: Configure Connector 1 by setting the connector name, scope, type, URL, and token.

Connector 1 Example:

- Name: MITRE ATT&CK Import Connector
- Scope: Imports ATT&CK knowledge into OpenCTI
- Type: Import connector
- URL: http: //<opencti-ip-or-hostname>
- Token: Dedicated connector token created in OpenCTI

The purpose for this connector is to bring MITRE ATT&CK knowledge into OpenCTI so that threats, tactics, and techniques can be viewed in a structured way.

Step 5: Configure Connector 2 by setting the connector name, scope, type, URL, and token.

Connector 2 Example:

- Name: MISP Enrichment Connector
- Scope: Enriches indicators with external threat intelligence
- Type: Enrichment connector
- URL: http://<opencti-ip-or-hostname>
- Token: Dedicated connector token created in OpenCTI

The purpose of this connector is to enrich indicators with extra context such as reputation or related intelligence so that the data in OpenCTI is more useful.

Step 6: Demonstrate basic usage by adding or viewing an indicator, a malware or campaign object, and a graph or relationship view.

Develop and Apply Risk Management Strategies:

Step 1: Identify risks from vulnerability scan results by reviewing the results of the vulnerability scan done earlier using Nmap. The scan results should be analyzed to determine:

- Exposed services
- Outdated software
- Unsecure configurations
- Unnecessary open ports
- Weak authentication mechanisms

Not only that, but each finding should be evaluated based on:

- The likelihood of exploitation
- The potential impact
- Exposure level
- The importance of the affected system

Step 2: From the results of the vulnerability scan, select two risks that would have the most severe impact on the system or network.

Step 3: Create a risk monitoring procedure:

1. Run vulnerability scans on a regular schedule to detect new security weaknesses or configuration changes.
2. Maintain a record of identified risks and track whether mitigation actions have been completed. This can be done using a vulnerability management system.
3. Services such as SSH should be continuously monitored to detect suspicious activity.
4. Regularly check for software updates and apply security patches ASAP to reduce the chance of being exposed to newly discovered vulnerabilities.
5. If new vulnerabilities are discovered or system configurations change, reassess risk to determine if more/different mitigation actions are needed.

Step 4: Document all risk assessments and mitigation decisions.

Implement Security Monitoring and Incident Response:

Step 1: Set up a basic security monitoring use case.

Example: Repeated failed logins from the same IP/account

Include:

- A log source such as (Ex. Linux authentication logs)
- A detection rule that watches for repeated failed logins
- An alert output that shows when the threshold is exceeded
- An explanation of why the event matters

Step 2: Define detection rules.

Example:

- If the same user account fails to log in 10 times within 3 minutes
- If the same IP has repeated failed logins across multiple accounts
- If the target account is an administrator account

Step 3: Create the alert prioritization process.

Example:

- High priority:
 - Repeated failed logins on an admin account
 - Multiple attempts from the same source in a short time
 - Evidence of a successful login after failed attempts
- Medium priority
 - Failed logins on a standard account
 - Unusual login activity that is not yet confirmed as malicious
- Low priority
 - A small number of failed logins that could be a typo
 - Other harmless activity

Step 4: Document response procedures.

Example for the failed login use case:

- Review the alert and confirm the source of the activity
- Check whether the account owner was trying to log in normally
- Determine whether the attempts came from a suspicious IP
- If the activity appears to be malicious, block the IP and/or temporarily lock the account
- Reset credentials if compromise is suspected
- Document the event for incident tracking and future review

Step 5: Create 1 incident response scenario, classify the incident, document the incident response steps, and write the lessons learned.

Example: Brute force login attempt against a Linux system

- Incident Classification (Example):
 - Incident type: Brute force login attempt
 - Severity: Medium or High depending on the account that was targeted
 - Impact: Possible unauthorized access
 - Confidence: Moderate or High if the pattern is repeated and clearly suspicious
- Document Incident Response Steps (Example):
 - Detect the suspicious login pattern via detection rules
 - Validate the alert by checking the logs and confirming the source
 - Contain the threat by blocking the suspicious IP and/or disabling the account
 - Eliminate the issue by resetting the password or removing unauthorized access
 - Restore normal access for the legitimate user and document the incident

- Write the Lessons Learned (Example):
 - Strong password policies are good for reducing the risk of a brute force working
 - MFA would make account compromise much harder
 - Repeated failed logins should be monitored and prioritized quickly
 - Clear log visibility helps the people responsible for cybersecurity react faster
 - Incident response is better when alerts and response steps are documented

Step 6: Show evidence of functionality.

Examples of evidence:

- Screenshot of detection rules
- Screenshot of alerts
- Log output showing failed login attempts
- Documentation of the response steps taken